Triggers and PL/pgSQL in PostgreSQL

Michel Ferreira

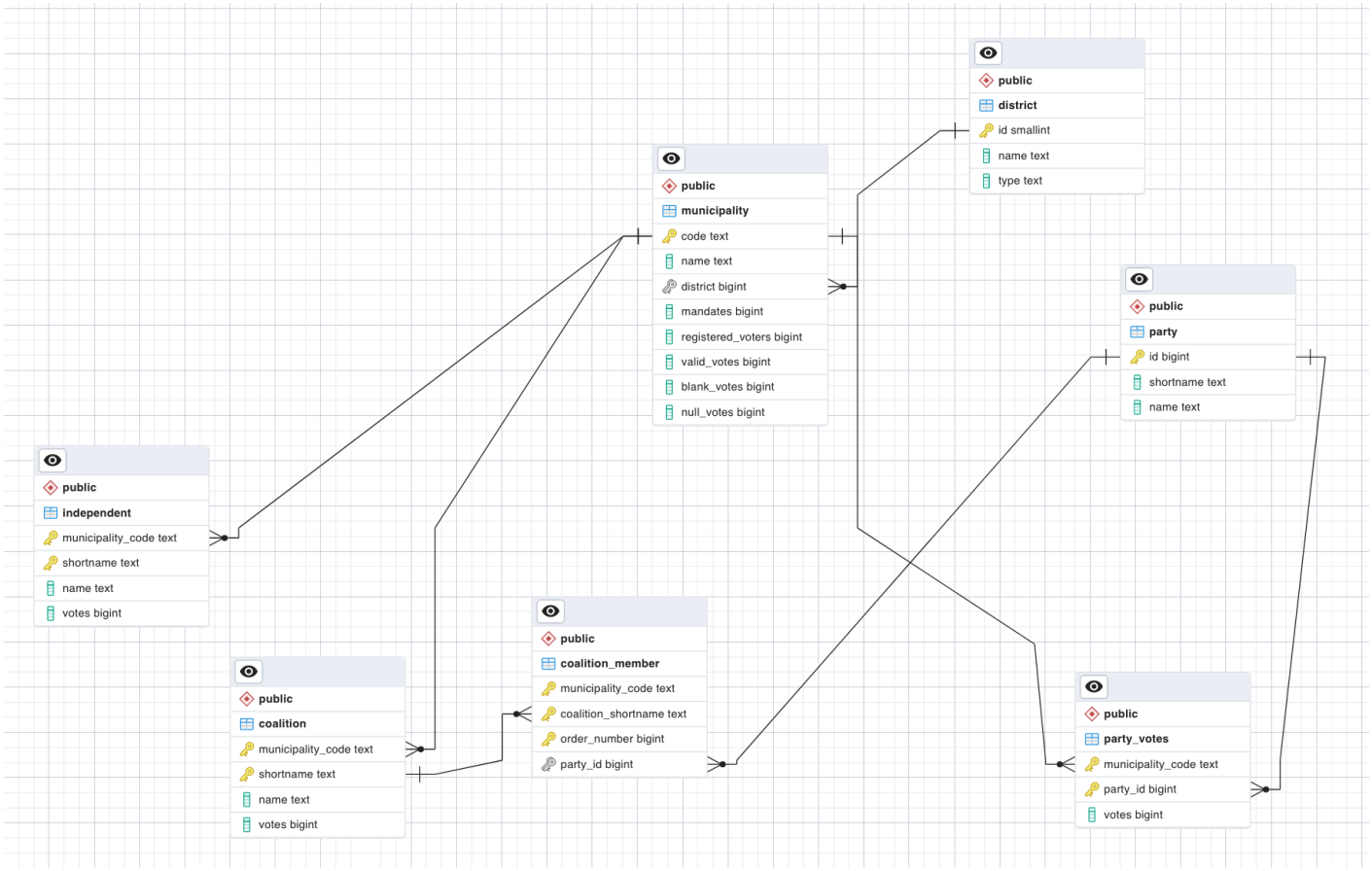
Advanced Topics in Databases

Why PL/pgSQL on top of plain SQL?

- **PostgreSQL term** for its procedural language is PL/pgSQL. People sometimes informally say “PL/SQL in Postgres”, but the PostgreSQL language name is PL/pgSQL.
- Plain SQL is excellent for set-based queries, joins, grouping, and updates.
- PL/pgSQL adds variables, assignments, conditional logic, loops, exceptions, and reusable functions.
- A trigger function is also written in PL/pgSQL, but it is called automatically by table events.
- The agenda for this class is: small function -> richer function -> trigger function -> CREATE TRIGGER.

Key idea: A PL/pgSQL function is still SQL-centric: it simply lets us orchestrate SQL statements with procedural control flow.

Autárquicas 2021 schema used in the examples



A first minimal function

- Every PostgreSQL function declares a return type, a body, and a language.
- The body is enclosed with dollar quoting, which keeps it readable and avoids escaping many quote characters.
- This first example is intentionally tiny: it simply returns a constant text value.
- Only after seeing this skeleton should we place real queries inside it.

```
CREATE FUNCTION hello_aut2021()
RETURNS text AS $$
BEGIN
  RETURN 'hello from aut2021';
END;
$$ LANGUAGE plpgsql;
```

Observation: BEGIN ... END encloses the function body. The final RETURN must match the declared return type, here text.

Anatomy of a PL/pgSQL function

- **CREATE FUNCTION** defines the function name and parameter list.
- **RETURNS** states the data type produced by the function.
- **DECLARE** is optional; it is where local variables are introduced.
- **BEGIN ... END** contains the executable statements.
- **LANGUAGE plpgsql** tells PostgreSQL which procedural language interprets the body.

```
CREATE FUNCTION demo_structure(p_code text)
RETURNS bigint AS $$
DECLARE
  v_mandates bigint;
BEGIN
  SELECT mandates
  INTO v_mandates
  FROM municipality
  WHERE code = p_code;

  RETURN v_mandates;
END;
$$ LANGUAGE plpgsql;
```

Why this example already matters: Even though it is still small, it already shows the most common pattern in PL/pgSQL: run a query, store the result in a variable, and return it.

Variables and assignment

- Variables are declared in the DECLARE section.
- We can give them an initial value immediately, using :=.
- A query can assign to variables with SELECT … INTO.
- When multiple columns are selected, multiple variables can be filled in one statement.
- COALESCE and NULLIF remain regular SQL expressions inside PL/pgSQL.

```
CREATE FUNCTION municipality_ballots(p_code text)
RETURNS bigint AS $$
DECLARE
  v_valid bigint := 0;
  v_blank bigint := 0;
  v_null  bigint := 0;
BEGIN
  SELECT valid_votes, blank_votes, null_votes
  INTO v_valid, v_blank, v_null
  FROM municipality
  WHERE code = p_code;

  RETURN COALESCE(v_valid, 0)
       + COALESCE(v_blank, 0)
       + COALESCE(v_null, 0);
END;
$$ LANGUAGE plpgsql;
```

Very common pattern: PL/pgSQL does not replace SQL expressions; it lets us compose and control them with variables.

Simple example: get the number of mandates

- This is the first realistic function over Autárquicas 2021 database.
- It reads the mandates column from municipality and returns one scalar value.
- The parameter p_code is used inside the WHERE clause like any other variable.
- The local variable v_mandates exists only while the function is running.

```
CREATE FUNCTION get_mandates(p_code text)
RETURNS bigint AS $$
DECLARE
  v_mandates bigint;
BEGIN
  SELECT mandates
  INTO v_mandates
  FROM municipality
  WHERE code = p_code;

  RETURN v_mandates;
END;
$$ LANGUAGE plpgsql;
```

Calling a PL/pgSQL function from SQL

- Once created, a PL/pgSQL function is called like any other SQL function.
- It can be called with a literal argument.
- It can also be called per row inside a query over municipality.
- The procedural part is hidden inside the function body; from the outside, this is regular SQL usage.

```
SELECT get_mandates('131200');

SELECT code,
       name,
       get_mandates(code) AS mandates_again
FROM municipality
ORDER BY code
LIMIT 10;
```

Important distinction: Creating a function is procedural work. Calling it later is normal SQL.

IF / ELSIF / ELSE with municipality data

- PL/pgSQL becomes useful when the returned result depends on a decision.
- Here we classify a municipality by participation level.
- The function first computes total ballots and a participation ratio.
- Then IF / ELSIF / ELSE chooses the returned label.

```
CREATE FUNCTION participation_level(p_code text)
RETURNS text AS $$
DECLARE
  v_registered bigint;
  v_ballots bigint;
  v_ratio numeric;
BEGIN
  SELECT registered_voters,
         valid_votes + blank_votes + null_votes
  INTO v_registered, v_ballots
  FROM municipality
  WHERE code = p_code;

  v_ratio := v_ballots::numeric / NULLIF(v_registered, 0);

  IF v_ratio >= 0.70 THEN
    RETURN 'high';
  ELSIF v_ratio >= 0.50 THEN
    RETURN 'medium';
  ELSE
    RETURN 'low';
  END IF;
END;
$$ LANGUAGE plpgsql;
```

Notice the mixture: The arithmetic is still SQL-like, but the decision logic is procedural.

IF / ELSIF / ELSE with municipality data

- Using the defined function, select the municipality names where participation was high.

```
SELECT name
FROM municipality
WHERE participation_level(code) = 'high'
ORDER BY name;
```

```
 name
-----------------------------
 AGUIAR DA BEIRA
 ALCOUTIM
 ALFÂNDEGA DA FÉ
 ALMODÔVAR
 ALTER DO CHÃO
 ARRONCHES
 AVIS
 BARCELOS
 BARRANCOS
 CASTANHEIRA DE PÊRA
 CASTELO DE PAIVA
 CASTELO DE VIDE
 CELORICO DE BASTO
 CORVO
 CRATO
 CUBA
 ÉVORA
 ...
 VILA NOVA DE CERVEIRA
(49 rows)
```

FOUND and a simple exception

- After SELECT … INTO, PostgreSQL sets the special boolean variable FOUND.
- FOUND is true when the query produced a row; otherwise it is false.
- This lets us fail early with a clear message if the requested key is absent.
- That is often more useful for teaching than returning NULL silently.

```
CREATE FUNCTION get_party_name_strict(p_id bigint)
RETURNS text AS $$
DECLARE
  v_name text;
BEGIN
  SELECT name
  INTO v_name
  FROM party
  WHERE id = p_id;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'Party id % does not exist', p_id;
  END IF;

  RETURN v_name;
END;
$$ LANGUAGE plpgsql;
```

Why this matters: PL/pgSQL can validate assumptions very explicitly, instead of letting incorrect inputs propagate quietly.

Looping over rows with FOR ... IN SELECT

- Sometimes a loop is useful, even when a pure SQL aggregate would also solve the task.
- The loop variable r is a record that represents the current row.
- In each iteration, the votes value is added to the running total.
- This example is deliberately simple and stays inside party_votes only.

```
CREATE FUNCTION sum_party_votes_loop(p_code text)
RETURNS bigint AS $$
DECLARE
  r record;
  v_total bigint := 0;
BEGIN
  FOR r IN
    SELECT votes
    FROM party_votes
    WHERE municipality_code = p_code
  LOOP
    v_total := v_total + COALESCE(r.votes, 0);
  END LOOP;

  RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

Pedagogical note: In production SQL, SUM(votes) would usually be shorter; here the goal is to introduce the loop construct clearly.

Returning a table with RETURNS TABLE and RETURN QUERY

- A PL/pgSQL function does not need to return only one scalar value.
- RETURNS TABLE declares the output columns.
- RETURN QUERY emits the rows of a SELECT as the function result.
- This example joins party_votes with party to obtain readable party short names.

```
CREATE FUNCTION list_party_votes(p_code text)
RETURNS TABLE(party_shortname text, votes bigint) AS $$
BEGIN
  RETURN QUERY
  SELECT p.shortname, pv.votes
  FROM party_votes pv
  JOIN party p
    ON p.id = pv.party_id
  WHERE pv.municipality_code = p_code
  ORDER BY pv.votes DESC;
END;
$$ LANGUAGE plpgsql;
```

Very useful distinction: RETURN sends back one scalar value; RETURN QUERY sends back the rows of a query result.

Before triggers: what we now know about PL/pgSQL functions

- A function can declare variables, assign values, run queries, make decisions, loop, raise exceptions, and return either a scalar result or a table result.
- So far, every function was called explicitly by us in a SQL statement.
- A trigger function is different only in one major respect: PostgreSQL calls it automatically when a table event occurs.
- That is why the next conceptual step is not more SQL syntax, but understanding the trigger mechanism.

PL/pgSQL and Triggers: Do not think of triggers as a different language. A trigger function is still PL/pgSQL; it simply runs under a special calling convention.

From ordinary functions to trigger functions

- **Ordinary function** we call it explicitly with SELECT, in an expression, or inside another function.
- **Trigger function** PostgreSQL calls it automatically because some INSERT, UPDATE, or DELETE event happened.
- **Ordinary function return types** text, bigint, boolean, table(...), void, and so on.
- **Trigger function return type** trigger, because PostgreSQL expects the function to obey trigger semantics.

Mental model: First learn the code that should run. Then learn the CREATE TRIGGER statement that says when and where that code must run.

Trigger function vs CREATE TRIGGER

- **CREATE FUNCTION … RETURNS trigger** creates the procedural code that will run.
- **CREATE TRIGGER** attaches that code to a table event.
- **The function answers** “What should happen?”
- **The trigger answers** “When, on which table, and for which operation should it happen?”

In one sentence: The trigger function is the behaviour; the trigger object is the event binding.

Why does a trigger function say RETURNS trigger?

- Every PostgreSQL function must declare a return type.
- For trigger functions, the special return type is trigger.
- This does not mean the function returns a visible “trigger object” to the user.
- It means PostgreSQL will call the function using the trigger protocol: special variables are available, and the return value controls or accompanies the row operation.
- For a BEFORE row-level trigger, RETURN NEW usually accepts the row, RETURN OLD is typical in DELETE, and RETURN NULL can cancel the row operation.

Note: RETURNS trigger tells PostgreSQL: “This function is compatible with the trigger mechanism, not just with ordinary SQL calls.”

Special trigger variables: NEW, OLD, and TG_OP

- **NEW** the new row values being inserted or updated.
- **OLD** the previous row values, available in UPDATE and DELETE.
- **TG_OP** the operation name, such as INSERT, UPDATE, or DELETE.
- These variables are not declared by us in DECLARE. PostgreSQL provides them automatically when the trigger fires.
- This is precisely why the same PL/pgSQL language feels different inside a trigger function.

Typical usage: Validation uses NEW; auditing often compares OLD and NEW; multi-operation trigger functions inspect TG_OP.

First trigger example: reject negative votes

- We start with the simplest useful rule: votes should not be negative.
- Because the row must be checked before it is written, this will be a BEFORE trigger.
- The trigger function inspects NEW.votes and raises an exception if the value is invalid.
- If the value is acceptable, it returns NEW.

```
CREATE FUNCTION check_nonnegative_party_votes()
RETURNS trigger AS $$
BEGIN
  IF NEW.votes < 0 THEN
    RAISE EXCEPTION 'Votes cannot be negative';
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Meaning of RETURN NEW here: Proceed with the insert or update, using the row currently stored in NEW.

Binding the trigger function to party_votes

- The previous slide defined only the code.
- This slide defines when that code will run.
- The trigger fires before INSERT or UPDATE of votes on party_votes.
- FOR EACH ROW means the function is executed once per affected row.

```
CREATE TRIGGER trg_check_nonnegative_party_votes
BEFORE INSERT OR UPDATE OF votes
ON party_votes
FOR EACH ROW
EXECUTE FUNCTION check_nonnegative_party_votes();
```

Separation of concerns: CREATE FUNCTION defines the logic. CREATE TRIGGER binds that logic to an event on a concrete table.

What happens when the trigger fires?

- If a statement tries to insert or update a negative votes value, the trigger function raises an exception and PostgreSQL rejects the row.
- If the value is valid, the row is returned unchanged and the operation continues.
- So the function body contains the rule, while the trigger definition determines its scope.

```
-- accepted
UPDATE party_votes
SET votes = 1200
WHERE municipality_code = 'some_code'
  AND party_id = 1;

-- rejected
UPDATE party_votes
SET votes = -10
WHERE municipality_code = 'some_code'
  AND party_id = 1;
```

Note: The exact municipality code is not important here. What matters is understanding that the trigger is checked automatically; we do not call the function ourselves.

BEFORE vs AFTER, and ROW vs STATEMENT

- **BEFORE** use it when the row may need to be validated, modified, or rejected before writing.
- **AFTER** use it when the row has already been written and we now want a side effect, such as logging or refreshing a summary.
- **FOR EACH ROW** run once per affected row. NEW and OLD are central here.
- **FOR EACH STATEMENT** run once for the whole SQL statement, not once per row.

Simple rule of thumb: Validation usually suggests BEFORE ROW triggers; audit logging and summary maintenance often suggest AFTER ROW triggers.

Audit example: support table

- Auditing means keeping a separate history of changes.
- The operational data remains in party_votes; the history is written into a small audit table.
- This support table is intentionally simple, but it is enough to illustrate OLD, NEW, and TG_OP together.

```
CREATE TABLE party_votes_audit (
  audit_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  municipality_code text NOT NULL,
  party_id bigint NOT NULL,
  old_votes bigint,
  new_votes bigint,
  changed_at timestamptz NOT NULL DEFAULT now(),
  operation text NOT NULL
);
```

Why a separate table?: Audit information usually has a different purpose from operational data: it records change history rather than the current state.

Audit trigger function

- This function inserts one audit row every time party_votes changes.
- COALESCE lets the code work across INSERT, UPDATE, and DELETE.
- OLD.votes is null in INSERT; NEW.votes is null in DELETE.
- The final return depends on the operation: DELETE returns OLD, otherwise we return NEW.

```
CREATE FUNCTION audit_party_votes()
RETURNS trigger AS $$
BEGIN
  INSERT INTO party_votes_audit(
    municipality_code, party_id, old_votes, new_votes, operation
  )
  VALUES (
    COALESCE(NEW.municipality_code, OLD.municipality_code),
    COALESCE(NEW.party_id, OLD.party_id),
    OLD.votes,
    NEW.votes,
    TG_OP
  );

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  ELSE
    RETURN NEW;
  END IF;
END;
$$ LANGUAGE plpgsql;
```

Notice how much context the trigger gets: The trigger function does not need extra parameters for the changed row; PostgreSQL already provides OLD and NEW.

Audit trigger creation

- Because logging is a side effect after the change, AFTER is a natural choice.
- The trigger is attached to INSERT, UPDATE, and DELETE on party_votes.
- FOR EACH ROW ensures one audit row per changed row.

```
CREATE TRIGGER trg_audit_party_votes
AFTER INSERT OR UPDATE OR DELETE
ON party_votes
FOR EACH ROW
EXECUTE FUNCTION audit_party_votes();
```

Interpretation: Whenever party_votes changes, PostgreSQL writes the audit record automatically. No application code needs to remember to do it.

A second trigger use case: maintain a simple summary table

- Suppose we want a small table with the total of party votes per municipality.
- This is a derived value, because it can always be recomputed from party_votes.
- Maintaining such a summary table is a classic trigger example.
- To keep the trigger logic readable, we will separate the refresh computation from the trigger itself.

Design choice: One function will recompute one municipality total. The trigger function will only decide which municipality code must be refreshed.

Support table and refresh function returning void

- The support table stores one row per municipality.
- The refresh function recomputes the sum from party_votes and upserts the result.
- RETURNS void is appropriate because the caller needs only the side effect, not a data value.

```
CREATE TABLE municipality_party_vote_total (
  municipality_code text PRIMARY KEY
    REFERENCES municipality(code),
  total_votes bigint NOT NULL
);

CREATE FUNCTION refresh_municipality_party_total(p_code text)
RETURNS void AS $$
BEGIN
  INSERT INTO municipality_party_vote_total(municipality_code, total_votes)
  SELECT p_code, COALESCE(SUM(votes), 0)
  FROM party_votes
  WHERE municipality_code = p_code
  ON CONFLICT (municipality_code)
  DO UPDATE SET total_votes = EXCLUDED.total_votes;
END;
$$ LANGUAGE plpgsql;
```

New construct: ON CONFLICT lets the helper function behave the same way whether the municipality summary row already exists or not.

Trigger function that calls the refresh helper

- Because refresh_municipality_party_total returns void, we call it with PERFORM.
- INSERT and DELETE affect one municipality code; UPDATE may affect two if the key changed.
- The trigger function therefore focuses on the event logic, not on the aggregate SQL itself.

```
CREATE FUNCTION trg_refresh_municipality_party_total()
RETURNS trigger AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    PERFORM refresh_municipality_party_total(OLD.municipality_code);
    RETURN OLD;
  ELSE
    PERFORM refresh_municipality_party_total(NEW.municipality_code);

    IF TG_OP = 'UPDATE'
       AND NEW.municipality_code <> OLD.municipality_code THEN
      PERFORM refresh_municipality_party_total(OLD.municipality_code);
    END IF;

    RETURN NEW;
  END IF;
END;
$$ LANGUAGE plpgsql;
```

Why PERFORM?: PERFORM runs a function when we care only about its side effect. Writing SELECT would suggest we intend to use a returned value.

Attaching the summary-maintenance trigger

- The summary must be refreshed after changes are already written to party_votes, so AFTER is appropriate.
- Again we use FOR EACH ROW, because each modified row may change the total of one municipality.

```
CREATE TRIGGER trg_refresh_municipality_party_total
AFTER INSERT OR UPDATE OR DELETE
ON party_votes
FOR EACH ROW
EXECUTE FUNCTION trg_refresh_municipality_party_total();
```

Result: The support table municipality_party_vote_total stays synchronized automatically as party_votes changes.

Putting the pieces together: complete workflow

- Validation trigger: prevents invalid negative vote values before data is written.
- Audit trigger: records what changed after the write succeeds.
- Summary trigger: refreshes a derived total after the write succeeds.
- All three are based on the same pattern: write a small trigger function first, then bind it with CREATE TRIGGER.

Final example: a virtual parliament with 230 deputies

- Now we move to one larger worked example, still based only on the aut2021 schema and one support mechanism.
- We will use only party_votes; coalitions and independents are intentionally ignored in this final example.
- Municipalities enter the calculation one by one, ordered by their total number of ballots, from the smallest to the largest.
- At each step, a trigger will refresh the current decomposition of 230 deputies by party, using the D'Hondt method.

What makes this a good example: It combines simple functions, a support table, a trigger, and a non-trivial SQL computation, but each individual piece is still understandable on its own.

Step 1: define the municipality order

- We need a deterministic order of municipalities by their total number of ballots.
- We can compute total ballots from valid_votes + blank_votes + null_votes.
- row_number() assigns step 1 to the smallest municipality, step 2 to the next, and so on.

```
CREATE VIEW municipality_step_order AS
SELECT code,
       name,
       valid_votes + blank_votes + null_votes AS total_ballots,
       row_number() OVER (
         ORDER BY valid_votes + blank_votes + null_votes, code
       ) AS step_no
FROM municipality;
```

Why a view?: The ordering is derived from municipality and does not need to be stored manually.

Step 2: a one-row state table and a result table

- The state table says how many municipalities are currently active in the virtual parliament experiment.
- The result table stores the current 230-deputy distribution per party.
- These are small support tables around the original aut2021 data.

```
CREATE TABLE virtual_parliament_state (
  id boolean PRIMARY KEY DEFAULT TRUE,
  current_step integer NOT NULL
);

CREATE TABLE virtual_parliament_result (
  party_id bigint PRIMARY KEY
    REFERENCES party(id),
  deputies integer NOT NULL
);
```

Interpretation: If current_step = 20, then only the first 20 municipalities in municipality_step_order are currently contributing to the virtual parliament result.

Refresh function - part I: choose municipalities and sum party votes

- The helper function will recompute the whole 230-seat result from the current step.
- First we select the municipalities whose step number is small enough.
- Then we aggregate party_votes over only those municipalities.

```
CREATE FUNCTION refresh_virtual_parliament(p_step integer)
RETURNS void AS $$
BEGIN
  DELETE FROM virtual_parliament_result;

  WITH RECURSIVE
  selected_municipalities AS (
    SELECT code
    FROM municipality_step_order
    WHERE step_no <= p_step
  ),
  selected_party_votes AS (
    SELECT pv.party_id,
           SUM(pv.votes) AS total_votes
    FROM party_votes pv
    JOIN selected_municipalities sm
      ON sm.code = pv.municipality_code
    GROUP BY pv.party_id
  ),
  divisors AS (
    SELECT 1 AS divisor
    UNION ALL
    SELECT divisor + 1
    FROM divisors
    WHERE divisor < 230
  ),
```

Notice the design: The function starts by deleting the previous decomposition. Then the CTEs build the new one from scratch for the requested step.

Refresh function - part II: D'Hondt quotients and top 230 seats

- Once party totals are known, D'Hondt is just a ranking of quotients total_votes / divisor.
- The top 230 quotients correspond to the 230 deputies.

```
 quotients AS (
    SELECT spv.party_id,
           spv.total_votes::numeric / d.divisor AS quotient
    FROM selected_party_votes spv
    CROSS JOIN divisors d
  ),
  top_230 AS (
    SELECT party_id
    FROM quotients
    ORDER BY quotient DESC, party_id
    LIMIT 230
  )
  INSERT INTO virtual_parliament_result(party_id, deputies)
  SELECT party_id, COUNT(*)::integer
  FROM top_230
  GROUP BY party_id;
END;
$$ LANGUAGE plpgsql;
```

Very important simplification: The D'Hondt method itself is expressed as plain SQL. PL/pgSQL is used only to package and trigger the recomputation.

Trigger function and trigger for the virtual parliament

- The state table changes rarely, so the trigger can be very small.
- Its only job is to call the refresh helper with the new current_step value.
- Because the helper returns void, the trigger function uses PERFORM.

```
CREATE FUNCTION trg_refresh_virtual_parliament()
RETURNS trigger AS $$
BEGIN
  PERFORM refresh_virtual_parliament(NEW.current_step);
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_virtual_parliament_refresh
AFTER INSERT OR UPDATE OF current_step
ON virtual_parliament_state
FOR EACH ROW
EXECUTE FUNCTION trg_refresh_virtual_parliament();
```

Again the same pattern: Complex SQL stays in a helper function; the trigger function remains tiny and easy to explain.

Using the final example in psql

- The first INSERT initializes the singleton state row.
- Later UPDATE statements move the experiment forward: step 1, step 20, step 100, and so on.
- After each change, we can inspect the party decomposition in virtual_parliament_result.

```
INSERT INTO virtual_parliament_state(id, current_step)
VALUES (TRUE, 1);

UPDATE virtual_parliament_state
SET current_step = 20
WHERE id = TRUE;

SELECT p.shortname, v.deputies
FROM virtual_parliament_result v
JOIN party p ON p.id = v.party_id
ORDER BY v.deputies DESC, p.shortname;
```

What you should notice: The trigger is invisible in the SELECT. It acts because we updated virtual_parliament_state, not because we called it directly.